

FIT3155

Advanced algorithms and data structures

Week 6 lecture notes by Taylor Kearney

Based on lecture slides by Arun Konagurthu



Faculty of Information Technology

November 6, 2025

Contents

1	Semi-numerical Algorithms	2
2	A closer look at basic arithmetic operations	2
	2.1 The complexity of basic addition and multiplication	3
	2.2 Fast integer multiplication	4
3	Congruence classes	7
4	Modular exponentiation	8
	4.1 Modular exponentiation by repeated squaring	9
5	Primality testing	13
	5.1 Naive primality testing	13
	5.2 Randomised primality testing	14

1 Semi-numerical Algorithms

Sources and recommended reading:

Cormen et al. Introduction to Algorithms (chapter 31)

Knuth. The Art of Computer Programming (Vol 2, chapter 4.3.3)

Mathematicians discover the perfect way to multiply [\[link\]](#)

We have already spent a significant portion of this unit examining algorithms that worked to manipulate the characters of a string. Given this previous focus, we would be remiss to not at least mention algorithms that work primarily with numeric data. To follow Knuth's second volume we have daubed this section *semi-numerical* algorithms, where the prefix *semi* allows us to encompass algorithms that walk the line between numeric and symbolic computation. These processes are fundamental to modern computing and involve everything from basic arithmetic such as addition and multiplication, to the generation of random numbers and the decomposition of large numbers into their prime factors. Given that the entire second volume of Knuth's *The Art of Computer Programming* is dedicated to this topic, along with an active field of research, it should go without saying that our introduction will be just a taste of what semi-numerical algorithms have to offer. However, this does not mean that our examination will be surface level in its entirety, as to properly appreciate the manipulation of numerical data we need to take a look under the hood.

2 A closer look at basic arithmetic operations

By this point in our computer science journey it is almost second nature to offhandedly ignore the contribution of basic arithmetic when analysing the complexity of an algorithm. You have likely claimed yourself that adding, or multiplying two numbers requires a constant amount of work and so we are typically concerned with how many of these operations our algorithm performs. That is we treat basic arithmetic as being an *atomic*, or indivisible operation, that always incurs the same overhead regardless of the numbers input. And for the most part this assumption is valid, but it is instructive to pause and think about whether this is always true.

So then, why is it that we consider operations like addition to be $O(1)$ -time? Well one way to justify this is to note that the addition of two numbers will (almost) always be in direct correspondence with the execution of a fixed number of assembly instructions in our machine. These instructions themselves can be decomposed into a fixed number of more primitive operations such as, register and memory transfers, as well as the use of some circuitry in our machine's *Arithmetic logic Unit* (assuming the inputs are integers). From this perspective it seems perfectly reasonable to claim that we can add two integers in constant time, but we're overlooking one crucial detail, these are all finite precision operations.

There is a limit to the size of the numbers that our computer can represent in a single *word* and although this depends on the architecture in question - 32 bit and 64 bit systems are common - we cannot circumvent this fundamental limitation. In the context of adding two numbers, this essentially means that the circuitry of our computer can only natively add two numbers together if they are small enough to be represented using two words of data. Thus, when working with numbers too large to be represented with a single word, we must decompose them into several words and develop methods to add each component individually before reconstructing the result. The important realisation here is that arithmetic with arbitrarily high precision is **not** a constant time operation!

Let's define some *primitive* operations that we can utilise to quantify the complexity of arithmetic with arbitrary precision. As we have discussed, our machines are able to perform basic arithmetic on two words in constant time. In particular, it is reasonable to assume that we have access to the following arithmetic operations:

- Addition, or subtraction of two one-place integers giving a one-place result and carry.
- Multiplication of two one-place integers giving a two-place result.
- Division of a two-place integer by a one-place integer yielding a one-place result (quotient) and a one-place remainder.

We assume that each of these operations require a constant amount of time and note that by 'one-place' integer we are referring to the analogy of a digit in a number system with an arbitrary base. For instance, in a base 10, or decimal system, the number 365 is a 3 place integer, i.e. it contains 3 digits; 3, 6 and 5. Similarly, if we work in binary, each place corresponds to a single bit and in our computer each place corresponds to a single word of data.

Question 1: Long multiplication

Argue that the number of primitive operations required to multiply two non-negative integers α and β , consisting of n and m digits respectively is $O(mn)$. You can adopt a similar approach to that taken in example [1](#) and assume that the computation is performed using long multiplication.

2.2 Fast integer multiplication

In the previous section you should have convinced yourself that the long multiplication method you learnt in school can be adapted into an algorithm that requires $O(mn)$ -time to multiply an m -place integer by an n -place integer. Given the simplicity of this method one might wonder whether we can do better. After all multiplication is an essential operation, and multiplying 2 n -place integers using this method would require $O(n^2)$ -time, which becomes increasingly prohibitive as the size of the integers grow. Fortunately, there are several more advanced multiplication techniques that offer better asymptotic performance, but it took until 1963 for the first of these - Karatsuba multiplication - to be discovered.

Karatsuba multiplication requires $O(n^{\log_2(3)})$ to multiply 2 n -place integers and you will prove this in tutorial 8. We will focus on Karatsuba multiplication owing to its simplicity, but we should note that since its inception several more efficient methods have been developed. For instance, in 1971 Arnold Schönhage and Volker Strassen gave an $O(n \log(n) \log[(\log(n))])$ -time algorithm that leveraged the *Discrete Fourier Transform* (DFT) (see Cormen et al.) and recently (2019) David Harvey proposed an $O(n \log(n))$ -time method. Despite the existence of these more efficient methods, Karatsuba multiplication is actually still of practical use, and Python actually implements Karatsuba's method to perform integer multiplication!

Karatsuba multiplication

Karatsuba's method is a divide and conquer approach and aims to reduce the problem of multiplying two n -place integers into several multiplications of smaller integers, plus to some other less expensive operations. For the remainder of this discussion we will assume that our integers are expressed in binary, although all the conclusions generalise trivially to other number systems. In addition, we will assume that the operands u and v are $2d$ -bit non-negative integers for the sake of convenience, although you should consider how to adapt the following to more general operands e.g. negative integers, integers with a different numbers of bits, etc.

To decompose the product uv we first divide each integer into their most and least significant halves such that,

$$u = (\underbrace{u_{2d-1}u_{2d-2}\cdots u_d}_{U_1}\underbrace{u_{d-1}u_{d-2}\cdots u_0}_{U_0})_{\text{base-2}}, \text{ and}$$

$$v = (\underbrace{v_{2d-1}v_{2d-2}\cdots v_d}_{V_1}\underbrace{v_{d-1}v_{d-2}\cdots v_0}_{V_0})_{\text{base-2}}.$$

In this representation we have, $u = 2^d U_1 + U_0$ and $v = 2^d V_1 + V_0$ and the product,

$$uv = (2^d U_1 + U_0)(2^d V_1 + V_0),$$

can be expanded as:

$$\begin{aligned} uv &= 2^{2d} U_1 V_1 + 2^d (U_0 V_1 + U_1 V_0) + U_0 V_0 \\ &= 2^{2d} U_1 V_1 + 2^d (U_1 V_1 - U_1 V_1 + U_0 V_1 + U_1 V_0 - U_0 V_0 + U_0 V_0) + U_0 V_0 \\ &= 2^{2d} U_1 V_1 + 2^d (U_1 V_1 - (U_1 - U_0)(V_1 - V_0) + U_0 V_0) + U_0 V_0 \\ &= (2^{2d} + 2^d) U_1 V_1 - 2^d (U_1 - U_0)(V_1 - V_0) + (2^d + 1) U_0 V_0. \end{aligned}$$

Recalling that U_0, U_1, V_0 and V_1 all contain d -bits, we can see that we have reduced the multiplication between 2 $2d$ -bit numbers into:

1. three d -bit multiplications,

- $U_1 V_1$
- $(U_1 - U_0)(V_1 - V_0)$
- $U_0 V_0$

2. plus some *left shifting*, and

3. some additions and subtractions.

The same decomposition can be applied recursively to each of these smaller products until we have reduced the problem to multiplying two 1-bit operands. At this point we can directly evaluate the product using our primitive multiplication and then traverse back up our stack evaluating successively larger products until we have evaluated the original product.

The time complexity of this method can be expressed using the recurrence relation,

$$T(2d) = 3T(d) + cd,$$

where $T(n)$ is the time required to multiply 2 n -bit integers and c is a positive constant. The term cd in the previous expression serves as an upper bound on the amount of time required to perform the left shifting, as well as the additions and subtractions that are required to evaluate the product. The fact that these are all linear time operations is central to the effectiveness of this method, and we note that terms such as,

$$(2^{2d} + 2^d) U_1 V_1,$$

should be expanded and evaluated as,

$$2^{2d}U_1V_1 + 2^dU_1V_1,$$

in order to properly exploit these less costly operations.

Example 2: Karatsuba multiplication

Let's compute $13 \times 14 = 182$ using Karatsuba multiplication. We could work in decimal, but to be consistent with the previous discussion let's express the operands in binary,

$$\begin{aligned} u &= 1101, \\ v &= 1110. \end{aligned}$$

In order to make use of the decomposition we must now split both u and v into their most and least significant halves respectively. For instance, we can see that $u = 1101 = 1100 + 0001$ and if we define the most significant half of u to be $U_1 = 11$ and the least significant half to be $U_0 = 01$ we obtain,

$$u = U_100 + U_0.$$

Since 2^dU_1 is equivalent to padding d zeros to the right-hand side of U_1 (referred to as left shifting) we can write,

$$\begin{aligned} u &= 13_{\text{base-10}} = (\underbrace{11}_{U_1} \underbrace{01}_{U_0})_{\text{base-2}} \\ &= 2^2(\textcolor{red}{11}) + \textcolor{blue}{01} \end{aligned}$$

and similarly,

$$\begin{aligned} v &= 14_{\text{base-10}} = (\underbrace{11}_{V_1} \underbrace{10}_{V_0})_{\text{base-2}} \\ &= 2^2(\textcolor{red}{11}) + \textcolor{blue}{10}. \end{aligned}$$

Now we simply substitute our expressions for U_1, U_0, V_1 and V_0 into our decomposition to obtain,

$$\begin{aligned} uv &= (2^{2d} + 2^d)U_1V_1 - 2^d(U_1 - U_0)(V_1 - V_0) + (2^d + 1)U_0V_0 \\ &= (2^4 + 2^2)(\textcolor{red}{11})(\textcolor{red}{11}) - 2^2(\textcolor{red}{11} - \textcolor{blue}{01})(\textcolor{red}{11} - \textcolor{blue}{10}) + (2^2 + 1)(\textcolor{blue}{01})(\textcolor{blue}{10}) \\ &= (2^4 + 2^2)(\textcolor{red}{11})(\textcolor{red}{11}) - 2^2(10)(01) + (2^2 + 1)(\textcolor{blue}{01})(\textcolor{blue}{10}), \end{aligned}$$

where in the last line we have evaluated the differences in the second term.

To proceed we apply the method recursively to compute the products $(\textcolor{red}{11})(\textcolor{red}{11})$, $(10)(01)$, and $(\textcolor{blue}{01})(\textcolor{blue}{10})$. For instance, to evaluate $(\textcolor{red}{11})(\textcolor{red}{11})$, we note that we are multiplying two 2-bit numbers ($2d = 2$) and that our operands can be decomposed as $x = y = 2^1(\textcolor{red}{1}) + \textcolor{blue}{1}$. Thus, we obtain,

$$\begin{aligned} xy &= (2^{2d} + 2^d)X_1Y_1 - 2^d(X_1 - X_0)(Y_1 - Y_0) + (2^d + 1)X_0Y_0 \\ &= (2^2 + 2^1)(\textcolor{red}{1})(\textcolor{red}{1}) - 2^1(\textcolor{red}{1} - \textcolor{blue}{1})(\textcolor{red}{1} - \textcolor{blue}{1}) + (2^1 + 1)(\textcolor{blue}{1})(\textcolor{blue}{1}) \\ &= (2^2 + 2^1)\textcolor{red}{1} - 2^1(0)(0) + (2^1 + 1)(\textcolor{blue}{1}) \text{ (we can multiply 1-bit numbers using primitive operations)} \\ &= 2^2(\textcolor{red}{1}) + 2^1(\textcolor{red}{1}) + 2^1(\textcolor{blue}{1}) + 1 \\ &= 0100 + 0010 + 0010 + 0001 \\ &= \textcolor{red}{1001}, \end{aligned}$$

and the computations for $(10)(01) = 0010$, and $(\textcolor{blue}{01})(\textcolor{blue}{10}) = \textcolor{blue}{0010}$ follow similarly.

Finally, we traverse back up our stack and use these results to evaluate,

$$\begin{aligned}
 uv &= (2^4 + 2^2)(11)(11) - 2^2(10)(01) + (2^2 + 1)(01)(10) \\
 &= (2^4 + 2^2)1001 - 2^2(0010) + (2^2 + 1)(0010) \\
 &= 10010000 + 100100 - 001000 + 001000 + 0010 \\
 &= 10110110,
 \end{aligned}$$

which is $13 \times 14 = 182$ in binary as required.

Question 2: Karatsuba multiplication

Complete example 2 by using Karatsuba multiplication to confirm that $(10)(01) = 0010$, and $(01)(10) = 0010$.

3 Congruence classes

Following our close examination of multiplication it would seem natural to turn our gaze to division and not surprisingly it is possible to adapt long division into an $O(mn)$ -time algorithm (assuming an $m + n$ -place dividend and an n -place divisor) just as we did with long multiplication. In addition, there also exist several fast integer division algorithms, but we will not explore them here. Instead we will concentrate on modular arithmetic in order to prepare for our examination of *primality testing*, in which we will examine our first *randomised algorithm*.

You have likely discussed the formal notion of divisibility in previous units, or perhaps even in high school, but we will restate the appropriate properties here.

Theorem 1: Division

For any integer a and any positive integer n , there exists unique integers q and r such that,

$$a = qn + r,$$

where $0 \leq r < n$. We call $q = \lfloor \frac{a}{n} \rfloor$ the *quotient* and $r = a \bmod n$ is called the *remainder*.

Dividing the set of integers ($\mathbb{Z}$) by a fixed integer n defines n equivalence classes called *congruence classes*, where the representative of each class is the remainder $0 \leq r < n$ left by the division.

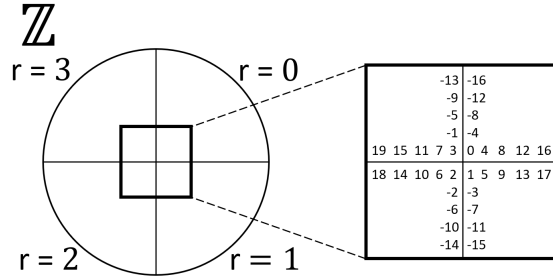
Definition 1: Congruence classes

Two integers a and b are said to be in the same congruence class modulo n if and only if the remainder when a is divided by n is the same as the remainder when b is divided by n . Equivalently we say that a and b are in the same congruence class modulo n if and only if $a - b$ is an integer multiple of n . This relationship is denoted as,

$$a \equiv b \pmod{n}.$$

Example 3: Congruence modulo 4

If we consider dividing each integer by 4, then we can partition the set of integers into 4 congruence classes with representatives 0, 1, 2 and 3 respectively, as shown below.



Note that each of our congruence classes contain infinitely many integers and can also contain negative numbers. For instance, 3 and -1 are in the same congruence class modulo 4 since $3 - (-1) = 4$, which is an integer multiple of 4.

4 Modular exponentiation

Modular exponentiation involves raising one number a to the power of another number b and determining the remainder when we divide the result by n , i.e. we need to evaluate,

$$a^b \bmod n.$$

This operation is used frequently in many number-theoretic algorithms and a naively we can perform it by first computing a^b and then modulo dividing the result by n to obtain the result. However, this naive approach is impractical when working with large numbers, particularly when b is large. To emphasise this let's analyse the time complexity of this naive approach.

Time complexity: Naive modular exponentiation (NOT EXAMINABLE)

Assume we are attempting to evaluate,

$$a^b \bmod n,$$

by first computing a^b through successive multiplications using the naive multiplication algorithm and then dividing by n to determine the remainder. Let d_a , d_b and d_n be the number of bits required to represent a , b , and n respectively.

To evaluate a^b we need to multiply a by itself b times. We begin by computing,

$$a^2 = a \times a,$$

which requires cd_a^2 operations, where c is a constant. Next we evaluate,

$$a^3 = a^2 \times a,$$

which requires $2cd_a^2$ operations, since a^2 is a $2d_a$ bit number. Similarly, evaluating

$$a^4 = a^3 \times a,$$

which requires $3cd_a^2$ operations, since a^3 is a $3d_a$ bit number. Continuing this sequence it would require,

$$cd_a^2(1 + 2 + 3 + \dots + b - 1) = cd_a^2 \frac{b(b-1)}{2} = O(d_a^2 b^2),$$

operations to compute a^b .

Finally, we need to determine $a^b \bmod n$, which requires dividing a^b by n and returning the remainder. If we assume we use long division and note that a^b is a bd_a -bit number this will require $O(bd_ad_n)$ -time. Thus, overall our naive modular exponentiation requires $O(d_a^2b^2 + bd_ad_n)$ -time. However, to better illustrate how inefficient this method is, it is best to express b in terms of d_b using the fact that,

$$\begin{aligned} d_b &= \lfloor \log_2(b) \rfloor + 1 \\ \implies b &= O(2^{d_b}). \end{aligned}$$

This result allows us to rewrite the time complexity of our method as $O(2^{2d_b}d_a^2 + 2^{d_b}d_ad_n)$ and we can see that it is exponential in d_b , the number of bits required to represent the exponent!

The previous analysis demonstrates that the size of the number a^b poses a significant obstacle for efficient modular exponentiation. In fact even if we had access to a linear time multiplication, or division algorithm, the naive algorithm would still require $O(2^{d_b})$ -time. To overcome this obstacle we need to design an algorithm that avoids dealing with such large numbers and to achieve this we will make use of the following property.

A useful property for modular exponentiation

If x , y and z are integers then,

$$xy \bmod z = ([x \bmod z] \times [y \bmod z]) \bmod z.$$

For instance, if $x = 7$, $y = 9$ and $z = 5$, then we have,

$$7 \times 9 \bmod 5 = 3,$$

and

$$([7 \bmod 5] \times [9 \bmod 5]) \bmod 5 = (2 \times 4) \bmod 5 = 8 \bmod 5 = 3,$$

as required.

Question 3: A property of modular arithmetic

Prove that,

$$xy \bmod z = ([x \bmod z] \times [y \bmod z]) \bmod z,$$

for integers x , y and z . You may find it helpful to begin by using definition 1 to interpret the left-hand-side of the expression.

We will now exploit this property to develop an efficient modular exponentiation algorithm.

4.1 Modular exponentiation by repeated squaring

Imagine that we are attempting to compute $7^{560} \bmod 561$. To leverage the fact that $xy \bmod z = ([x \bmod z] \times [y \bmod z]) \bmod z$ we need to decompose 7^{560} into the product of two or more integers. If we express the exponent in a binary representation we obtain,

$$560 = 1000110000_2,$$

and so we can write,

$$\begin{aligned} 7^{560} &= 7^{1 \times 2^9 + 0 \times 2^8 + 0 \times 2^7 + 0 \times 2^6 + 1 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 0 \times 2^0} \\ &= (7^{2^9})^1 (7^{2^8})^0 (7^{2^7})^0 (7^{2^6})^0 (7^{2^5})^1 (7^{2^4})^1 (7^{2^3})^0 (7^{2^2})^0 (7^{2^1})^0 (7^{2^0})^0 \\ &= 7^{2^9} 7^{2^5} 7^{2^4}. \end{aligned}$$

Applying our modular arithmetic property to this decomposition we obtain,

$$\begin{aligned} 7^{560} \bmod 561 &= 7^{2^9} 7^{2^5} 7^{2^4} \bmod 561 \\ &= \left(7^{2^9} \bmod 561 \times \left[7^{2^5} 7^{2^4} \right] \bmod 561 \right) \bmod 561 \\ &= \left(7^{2^9} \bmod 561 \times \left\{ \left[7^{2^5} \bmod 561 \times 7^{2^4} \bmod 561 \right] \bmod 561 \right\} \right) \bmod 561, \end{aligned}$$

and we can see that only the terms that correspond to there being a 1 in the binary representation of the exponent need to be considered. To efficiently compute these terms we note that,

$$\begin{aligned} 7^{2^i} \bmod 561 &= \left(7^{2^{i-1}} \times 7^{2^{i-1}} \right) \bmod 561 \\ &= \left(\left[7^{2^{i-1}} \bmod 561 \right] \times \left[7^{2^{i-1}} \bmod 561 \right] \right) \bmod 561, \end{aligned}$$

and so we can construct the required terms through *repeated squaring*.

Repeated squaring

Start by computing,

$$7^{2^0} \bmod 561 = 7.$$

Since,

$$7^{2^1} \bmod 561 = \left(\left[7^{2^0} \bmod 561 \right] \times \left[7^{2^0} \bmod 561 \right] \right) \bmod 561,$$

and we know that $7^{2^0} \bmod 561 = 7$, we must have,

$$\begin{aligned} 7^{2^1} \bmod 561 &= \left(\left[7^{2^0} \bmod 561 \right] \times \left[7^{2^0} \bmod 561 \right] \right) \bmod 561 \\ &= (7 \times 7) \bmod 561 \\ &= 49 \bmod 561 \\ &= 49. \end{aligned}$$

Similarly, we have,

$$\begin{aligned} 7^{2^2} \bmod 561 &= \left(\left[7^{2^1} \bmod 561 \right] \times \left[7^{2^1} \bmod 561 \right] \right) \bmod 561 \\ &= (49 \times 49) \bmod 561 \\ &= 49^2 \bmod 561 \\ &= 157, \end{aligned}$$

and so on.

Using the method of repeated squaring, we can compute $7^{2^i} \bmod 561$ for $1 \leq i \leq 9$ by simply squaring the previous result ($7^{2^{i-1}} \bmod 561$) and modulo dividing the result by 561. We can then use the appropriate terms in this sequence to evaluate $7^{560} \bmod 561$. Specifically, we set *result* = 1 initially and then apply the update,

$$\text{result} = (\text{result} \times 7^{2^i} \bmod 561) \bmod 561,$$

whenever the binary representation of the exponent has a 1 at position *i*, i.e. when $i \in \{4, 5, 9\}$. This process

will yield the desired result ($7^{560} \bmod 561 = 1$) as shown below,

$$7^{560} \bmod 561 = \left(\overbrace{\left(\overbrace{\left(\overbrace{7^{2^9} \bmod 561}^{\text{update 3}} \times \left\{ \left[\overbrace{7^{2^5} \bmod 561}^{\text{update 2}} \times \underbrace{7^{2^4} \bmod 561}_{\text{update 1}} \right] \bmod 561 \right\}} \right) \bmod 561 \right)} \right) \bmod 561.$$

The method of repeated squaring can be used to efficiently perform modular exponentiation and the pseudocode below describes the process required to evaluate $a^b \bmod n$.

```

1  modularExponentiation(a,b,n):
2      # Determine the binary representation of the exponent
3      binaryRep = getBinaryRepresentation(b)
4      # Base case
5      current = a mod n
6      if LSB in binaryRep == 1:
7          result = current
8      else:
9          result = 1
10     # Iterate through the remaining bits in the binary representation in reverse
11     for remaining bits in rev(binaryRep):
12         # Compute the next term in the sequence
13         current = (current*current) mod n
14         if currentBit == 1:
15             # update the result
16             result = (result * current) mod n
17     return result

```

In the pseudocode above we start by retrieving the binary representation of the exponent b on line 3 and we then initialise the result depending on whether the least significant bit (LSB), i.e. the right most bit, is set. To compute the remaining terms in the sequence we iterate through the remaining bits in the `binaryRep` in reverse order (right-to-left) computing the next term in the sequence and updating the result where necessary.

For instance, if we were computing $2^{12} \bmod 3 = 1$, we would have `binaryRep` = 1100, and on line 6 we would first check the right most 0 in `binaryRep` and set `result` = 1. Then in the loop we would iterate through the remaining three bits in a right-to-left fashion meaning that `currentBit` would be 0, then 1, and finally 1, and we would make updates to the result on the 2nd and 3rd iterations.

Let's round out our discussion of modular exponentiation by repeated squaring by determining how much of an improvement it offers when compared to the naive algorithm.

Time complexity: Modular exponentiation by repeated squaring (NOT EXAMINABLE)

Imagine that we wish to evaluate,

$$a^b \bmod n,$$

using repeated squaring. Let d_a , d_b and d_n be the number of bits required to represent a , b , and n respectively, and let $x_i = a^{2^i} \bmod n$ for $0 \leq i < d_b$.

We begin by evaluating,

$$a^{2^0} \bmod n = a \bmod n,$$

which requires $O(d_a d_n)$ -time assuming we are using long division to evaluate the modulo division by n . Following this, every term in the sequence will be computed by squaring the previous term and modulo dividing by n , i.e. $a^{2^i} \bmod n$ is evaluated as,

$$a^{2^i} \bmod n = (a^{2^{i-1}} \bmod n)^2 \bmod n.$$

Since $0 \leq x_{i-1} \leq n-1$ (as $x_{i-1} = a^{2^{i-1}} \bmod n$) we require at most d_n -bits to represent it and computing x_{i-1}^2 requires $O(d_n^2)$ -time in the worst case. Similarly, x_{i-1}^2 requires at most $2d_n$ -bits to represent and so in the worst case we require $O(2d_n^2)$ -time to evaluate the modulo division by n in $x_i = (x_{i-1}^2 \bmod n)$. Since the binary representation of b contains d_b bits we require,

$$\underbrace{O(d_a d_n)}_{\text{evaluating } x_0} + \underbrace{O(d_b d_n^2)}_{\text{squaring previous results}} + \underbrace{O(2d_b d_n^2)}_{\text{modulo dividing by } n} = O(d_a d_n + d_b d_n^2),$$

time to evaluate x_i for all $0 \leq i < d_b$. Finally, we note that the cost of updating the result to obtain $a^b \bmod n$ requires at most $O(d_n^2 + 2d_n^2) = O(d_n^2)$ for each 1 in the binary representation of b - of which there can be at most d_b - giving $O(d_b d_n^2)$ in the worst case.

In summary, the time complexity of evaluating $a^b \bmod n$ by repeated squaring is $O(d_a d_n + d_b d_n^2)$, which is a drastic improvement on the naive method which requires $O(2^{2d_b} d_a^2 + 2^{d_b} d_a d_n)$ time. Intuitively this improvement originates from the fact that by successively applying modulo division by n we prevent our operands from requiring any more than $2d_n$ -bits to represent. In contrast the naive method must operate on numbers that require up to $O(b d_a) = O(2^{d_b} d_a)$ bits to represent.

Armed with an efficient modular exponentiation algorithm we are now equipped to examine its application to *primality testing*.

5 Primality testing

Definition 2: Prime numbers

A prime number is a natural number - i.e. $\{1,2,3,\dots\}$ - strictly greater than 1 that has **no** positive divisors (factors) other than 1 and itself.

There are 25 primes in the first 100 natural numbers,

```
2  3  5  7 11
13 17 19 23 29
31 37 41 43 47
53 59 61 67 71
73 79 83 89 97
```

You can include 0 in the set of natural numbers if you'd like, it doesn't actually make any difference here.

Modern cryptography systems rely on the ability to generate large random prime numbers quickly so that they can be used as *keys* that help to secure information. For example, RSA (a popular public key algorithm) relies in part on the fact that we cannot efficiently factorise large numbers into their prime factors, and the larger we make the number the more difficult this problem becomes. However, we cannot simply choose a large number at random if we are to construct a useful communication protocol.

Instead we construct a large number n by choosing two prime numbers p and q at random and computing $n = pq$. This product along with the primes themselves are used to compute two keys; a public key that can be published and a private key that is kept secret. Messages are then encrypted using the public key and can only be decrypted using the corresponding private key. An attacker would need to reverse engineer the secret private key from the public key in order to read the encrypted message and the key generation method ensures that to do so would require determining the prime factors of n (p and q). Although the exact details of schemes like RSA are beyond the scope of this course, the previous discussion motivates further examination of primality testing.

Definition 3: Primality testing

In primality testing our task is to determine if a given number (that can be arbitrarily long) is prime.

5.1 Naive primality testing

Suppose that we wish to determine whether some natural number n is prime. The naive approach to test the primality of n would be to attempt to find a non-trivial - i.e. not 1, or n - factor of n . To do so is simple, we just divide n by $k \in [2, n-1]$ and check if it leaves a remainder of 0. If it does, then it must be divisible by k and since $k \neq \{1, n\}$, n must be *composite*. If n is not divisible by any $k \in [2, n-1]$, then the only factors it has are 1 and itself and we conclude that n must be prime. This *trial division* method is described by the pseudocode below.

```
1  trialDivision(n):
2      for k from 2 to n-1:
3          if n mod k == 0:
4              # n is divisible by k
5              return Composite
6      # n was not divisible by any k in [2,n-1]
7      return Prime
```

Time complexity: Trial division

Let n be a natural number whose binary representation contains d_n bits. To test the primality of n using the trial division algorithm presented above we would perform $n - 2$ divisions, and in the worst case each of these divisions requires $O(d_n^2)$ -time (assuming a basic division algorithm). This gives an overall time complexity of $O(nd_n^2)$ and recalling that $d_n = \lfloor \log_2(n) \rfloor + 1$ this can be written as $O(2^{d_n} d_n^2)$.

Currently the recommended length for RSA keys is 2048 bits and the primes used in key generation are approximately half that length at 1024 bits. Obviously testing the primality of 1024 bit number using trial division is not an attractive proposition and we need to develop a more efficient approach. We can actually improve our trial division algorithm by realising that we only need to test for factors up to $k = \sqrt{n}$, but the complexity of this improved method is still $O(\sqrt{n}d_n^2) = O(2^{\frac{d_n}{2}} d_n^2)$ and so we clearly need to adopt a new approach.

5.2 Randomised primality testing

The trial division test exploits the fact that prime numbers have no non-trivial factors, but unfortunately it yields an impractical algorithm. Fortunately, there are other properties of prime numbers that can be used as the basis for a primality test, and we will examine two in particular that form the foundation for the very practical *Miller-Rabin* primality test.

Theorem 2: Fermat's little theorem

If p is a prime number, then for every integer a , the number $a^p - a$ is an integer multiple of p , i.e.

$$a^p \equiv a \pmod{p}.$$

Equivalently, if a is not divisible by p , then we have,

$$a^{p-1} \equiv 1 \pmod{p}.$$

Proof (NOT EXAMINABLE):

Assume that p is prime and that a is an integer that is not divisible by p . Now consider the product of the non-zero multiples of a up to $(p-1)a$,

$$a \times 2a \times 3a \times \dots \times (p-1)a = (p-1)! \times a^{p-1},$$

under modulo division with p ,

$$a \times 2a \times 3a \times \dots \times (p-1)a \pmod{p} = (p-1)!a^{p-1} \pmod{p}. \quad (1)$$

Distributing the modulo p division inside the sequence on the left hand side of equation 1, we obtain:

$$[a \pmod{p} \times 2a \pmod{p} \times 3a \pmod{p} \times \dots \times (p-1)a \pmod{p}] \pmod{p}.$$

Notice that there are $p-1$ terms, each of the form $i \times a \pmod{p}$, for $1 \leq i \leq p-1$. Each term $i \times a \pmod{p}$ will yield some remainder r_i , where $1 \leq r_i \leq p-1$. This is because neither a nor i is divisible by p , and so $i \times a$ will never leave a remainder of 0.

Now let's consider whether any two terms - $i \times a \pmod{p}$ and $j \times a \pmod{p}$, where $1 \leq (i \neq j) \leq p-1$ - in the above product can leave the same remainder. In other words, can two or more of $p-1$ terms be in the same congruence class? Let us start by assuming they are in the same congruence class so that,

$$\begin{aligned} i \times a &\equiv j \times a \pmod{p} \\ \implies i \times a - j \times a &\equiv 0 \pmod{p} \\ \implies (i - j)a &\equiv 0 \pmod{p}. \end{aligned}$$

For this to be true, p must divide either a or $i - j$, but both of these are impossible since a is not divisible by p and $i - j$ cannot be divisible by p because $1 \leq (i \neq j) \leq p - 1$. Thus, we arrive at a contradiction and $i \times a \pmod{p}$ cannot be in the same congruence class modulo p as $j \times a \pmod{p}$,

$$i \times a \not\equiv j \times a \pmod{p}.$$

There was nothing special about i and j , so the previous statement is true for any $1 \leq (i \neq j) \leq p - 1$ in the product of terms:

$$[a \pmod{p} \times 2a \pmod{p} \times 3a \pmod{p} \times \dots \times (p-1)a \pmod{p}] \pmod{p}.$$

This means that each term has to yield a **different** remainder when divided by p , i.e. $r_i \neq r_j$ for $i \neq j$. But each remainder must be between 1 and $p - 1$, which implies that over all the $p - 1$ terms, we will have to see all possible $p - 1$ remainders between $[1, p - 1]$.

Therefore we must have,

$$\begin{aligned} [a \pmod{p} \times 2a \pmod{p} \times 3a \pmod{p} \times \dots \times (p-1)a \pmod{p}] \pmod{p} &= [1 \times 2 \times 3 \times \dots \times (p-1)] \pmod{p}. \\ \implies [a \pmod{p} \times 2a \pmod{p} \times 3a \pmod{p} \times \dots \times (p-1)a \pmod{p}] \pmod{p} &= (p-1)! \pmod{p}, \end{aligned}$$

and substituting this into equation 1 yields,

$$(p-1)! \pmod{p} = (p-1)! \times a^{p-1} \pmod{p},$$

which implies,

$$(p-1)!a^{p-1} \equiv (p-1)! \pmod{p}.$$

Thus, p must divide $(p-1)!a^{p-1} - (p-1)!$, giving,

$$(a^{p-1} - 1)(p-1)! \equiv 0 \pmod{p},$$

which means that p must divide $(p-1)!$, or $(a^{p-1} - 1)$. However, since p is prime it must be coprime to $(p-1)!$ (meaning the greatest common divisor is 1, $\gcd[(p-1)!, p] = 1$) and so p must divide $(a^{p-1} - 1)$, giving,

$$a^{p-1} \equiv 1 \pmod{p},$$

as required.

Multiplying both sides of this congruence by a yields the equivalent statement,

$$a^p \equiv a \pmod{p},$$

and so $a^{p-1} \equiv 1 \pmod{p} \implies a^p \equiv a \pmod{p}$. Moreover, if a is divisible by p we have,

$$a \equiv 0 \pmod{p},$$

and since $a \equiv b \pmod{n} \implies a^c \equiv b^c \pmod{n}$, we obtain,

$$a^p \equiv a \pmod{p}.$$

Finally since,

$$ca \equiv cb \pmod{n} \implies a \equiv b \pmod{n},$$

when c is coprime to n ($\gcd(c, n) = 1$); if we begin with,

$$a^p \equiv a \pmod{p},$$

and a is not divisible by p we must have,

$$a^{p-1} \equiv 1 \pmod{p}. \quad \blacksquare$$

Fermat's little theorem gives a necessary condition for n to be prime. It says that if n is prime it must satisfy,

$$a^{n-1} \equiv 1 \pmod{n},$$

for any a that is not divisible by n . Thus, we can use it to test the primality of n ! We simply compute $a^{n-1} \pmod{n}$ and check whether it is 1. If it is, then we declare n to be prime, otherwise we know that n must be composite. This idea is summarised by the pseudocode below.

```

1  fermatRandomisedTest(n):
2      a = choose a random number in the range 1 < a < n-1
3      if a^{n-1} mod n != 1:
4          # n doesn't satisfy Fermat's little theorem
5          return Composite
6      # n satisfies Fermat's little theorem so probably prime
7      return Prime

```

Notice that we are selecting a random *witness* a to testify to the compositeness of n and this is our first example of a randomised algorithm. In addition, note that we select $a \in (1, n-1)$, which not only guarantees that a is not divisible by n , but ensures that our witness is useful since if $a \in \{1, n-1\}$ we will always have,

$$a^{n-1} \equiv 1 \pmod{n},$$

regardless of whether n is prime or not.

Question 4: The useless witnesses

Given any odd natural number n , show that if $a = 1$, or $a = n - 1$ then,

$$a^{n-1} \equiv 1 \pmod{n}.$$

Unfortunately, Fermat's little theorem is not a sufficient condition for n to be prime. This means that if n is a number that satisfies,

$$a^{n-1} \equiv 1 \pmod{n},$$

it **does not** guarantee that n is prime. In other words, there are composite numbers - called *pseudoprimes* - that satisfy this condition for certain values of a and will fool our test. The witnesses a for which this occurs are called *Fermat's liars* and we say that n is a pseudoprime to the base a .

Due to this the above algorithm tests for primality probabilistically and it can incorrectly declare a composite number to be prime. However, it is important to note that it will never wrongly classify a prime number and so if it declares n to be composite it is guaranteed to be so.

Example 4: Fermat's little theorem and pseudoprimes

Let's test the primality of $n \in \{17, 9, 561\}$, using Fermat's little theorem.

1. Since 17 is prime it will pass our test for any $a \in [2, 15]$ and just as an example let's consider $a = 13$. Computing $13^{16} \pmod{17}$ using repeated squaring gives the sequence,

$$\begin{aligned}
 13^{2^0} \pmod{17} &= 13 \pmod{17} = 13 \\
 13^{2^1} \pmod{17} &= 13^2 \pmod{17} = 169 \pmod{17} = 16 \\
 13^{2^2} \pmod{17} &= 16^2 \pmod{17} = 256 \pmod{17} = 1 \\
 13^{2^3} \pmod{17} &= 1^2 \pmod{17} = 1 \\
 13^{2^4} \pmod{17} &= 1^2 \pmod{17} = 1
 \end{aligned}$$

and since $16_{10} = 10000_2$ we only need to consider the last term which yields $13^{16} \bmod 17 = 1$ as expected.

2. 9 is composite and if we select $a = 3$ we must compute,

$$3^8 \bmod 9 = 9 \times 3^6 \bmod 9 = 0,$$

and we can see that our test would declare 9 to be composite in this case.

3. Finally, let's consider $n = 561$ and choose our witness to be $a = 7$. We have actually already computed $7^{560} \bmod 561 = 1$ in our repeated squaring section and we can see that our test would conclude that 561 is prime. However, 561 is actually composite and can be factorised as $561 = 3 \times 11 \times 17$. Thus, 561 is a pseudoprime to the base 7! In fact, 561 is the first in a special class of pseudoprimes called *Carmichael numbers* which are pseudoprimes to the base a for all a such that $\gcd(n, a) = 1$. If we use $a = 3$ as our witness instead we find,

$$\begin{aligned} 3^{2^0} \bmod 561 &= 3 \\ 3^{2^1} \bmod 561 &= 3^2 \bmod 561 = 9 \\ 3^{2^2} \bmod 561 &= 9^2 \bmod 561 = 81 \\ 3^{2^3} \bmod 561 &= 81^2 \bmod 561 = 390 \\ 3^{2^4} \bmod 561 &= 390^2 \bmod 561 = 69 \\ 3^{2^5} \bmod 561 &= 69^2 \bmod 561 = 273 \\ 3^{2^6} \bmod 561 &= 273^2 \bmod 561 = 477 \\ 3^{2^7} \bmod 561 &= 477^2 \bmod 561 = 324 \\ 3^{2^8} \bmod 561 &= 324^2 \bmod 561 = 69 \\ 3^{2^9} \bmod 561 &= 69^2 \bmod 561 = 273, \end{aligned}$$

and recalling that $560_{10} = 1000110000_2$ we have,

$$3^{560} \bmod 561 = 273 \times \{(273 \times [69]) \bmod 561\} \bmod 561 = 375,$$

and we see that 561 is indeed composite.

Since there exist pseudoprimes to particular bases the obvious way to improve the accuracy of our primality test is to consider a collection of witnesses before declaring n to be prime. This reduces the likelihood of us wrongly classifying a composite as prime, but devolves into a randomised search for the factors of n when n is a Carmichael number. This is because Carmichael numbers are Fermat pseudoprimes for all witnesses a such that $\gcd(n, a) = 1$ and so will fool our test unless we are lucky and select a non-trivial factor of n as our witness. In light of this we need to leverage a second property of prime numbers to help distinguish the Fermat pseudoprimes from the actual primes.

Theorem 3: Square roots of unity (mod p)

If p is an odd prime then $x^2 \equiv 1 \pmod{p}$ has only two solutions, namely $x \equiv \pm 1 \pmod{p}$.

To develop a primality test based on theorem 3, it is best to first consider computing $a^{n-1} \pmod{n}$ using repeated squaring. Now we could use our regular repeated squaring algorithm, but we can make things a little nicer for ourselves if restrict the values n can take.

Since 2 is the only even prime, it is trivial to test the primality of all other even numbers. Thus, we will treat 2 as a special case and only consider odd inputs to our primality test from this point forward. Under this assumption we can make use of the following observation.

Miller-Rabin observation 1

If n is an odd number, then $n - 1$ must be even and thus can be represented as,

$$n - 1 = 2^s \times t,$$

where t is some odd integer.

For instance, if $n = 13$, then we have $n - 1 = 12$ and so $s = 2$, $t = 3$, giving,

$$n - 1 = 2^2 \times 3 = 12.$$

To see why this works consider the corresponding binary representations of $n - 1 = 12 = 1100_2$ and $t = 3_{10} = 11_2$, which gives,

$$n - 1 = 2^2 \times 3_{10} = 2^2(11_2) = 1100_2 = 12.$$

Using the above decomposition we have,

$$a^{n-1} \bmod n = a^{2^s \times t} \bmod n.$$

Now imagine computing $a^{n-1} \bmod n$ using repeated squaring, but instead of starting with $a^{2^0} \bmod n$, begin the sequence with $a^{2^0 \times t} \bmod n$, giving,

$$\begin{aligned} a^{2^0 \times t} \bmod n &= x_0 \\ a^{2^1 \times t} \bmod n &= x_0^2 \bmod n = x_1 \\ a^{2^2 \times t} \bmod n &= x_1^2 \bmod n = x_2 \\ &\vdots \\ a^{2^s \times t} \bmod n &= x_{s-1}^2 \bmod n = x_s. \end{aligned}$$

This looks deceptively similar to the bulk standard modular exponentiation by repeated squaring, but if we examine the last term in the sequence carefully, we can see that it is $x_s = a^{2^s \times t} \bmod n$, which we know to be equal to $a^{n-1} \bmod n$. So by exploiting the fact that $n - 1$ must be even, we can compute $a^{n-1} \bmod n$ without even needing to keep track of the 1s in the binary representation of $n - 1$! But that's not all, since we compute each term x_i in the sequence as,

$$x_i = x_{i-1}^2 \bmod n,$$

by theorem 3 we know that if n is an odd prime, and $x_i = 1$ - implying that $x_{i-1}^2 \equiv 1 \pmod n$ - then we must have, $x_{i-1} \equiv \pm 1 \pmod n$. This insight is the basis of our new primality test and when used in combination with the test based on Fermat's little theorem we arrive at the Miller-Rabin primality test.

The Miller-Rabin primality test

Given an odd natural number $n > 3$ and a witness $a \in [2, n - 2]$, the Miller-Rabin primality test proceeds as follows.

First find s and t such that $n - 1 = 2^s \times t$ and compute the sequence,

$$\begin{aligned} x_0 &= a^{2^0 \times t} \bmod n \\ x_1 &= a^{2^1 \times t} \bmod n = x_0^2 \bmod n \\ x_2 &= a^{2^2 \times t} \bmod n = x_1^2 \bmod n \\ &\vdots \\ x_s &= a^{2^s \times t} \bmod n = x_{s-1}^2 \bmod n. \end{aligned}$$

using repeated squaring. Once we have computed $\langle x_0, x_1, \dots, x_s \rangle$, we simply run our test based on Fermat's little theorem. Recalling that,

$$x_s = a^{2^s \times t} \bmod n = a^{n-1} \bmod n,$$

we simply need to check whether $x_s = 1$; if it isn't then we return 'composite', otherwise n is either prime or a Fermat pseudoprime to the base a and need to run our sequence test.

Given that we know that $x_s = 1$ at this point, we must have $x_{s-1} = \pm 1$ if n is prime by theorem 3. Applying this logic to the other terms in our sequence $\langle x_0, x_1, \dots, x_s \rangle$, if we find $x_i = 1$ and $x_{i-1} \not\equiv \pm 1 \pmod{n}$ (for $0 < i \leq s$) then n **cannot** be prime by theorem 3 and we return composite.

Before proceeding is it important to address a subtle point that often leads to confusion when attempting to implement this primality test. Our sequence test requires us to test each term x_i to determine if,

$$x_i = 1.$$

If it is, we must then check if the previous term (x_{i-1}) in the sequence satisfies,

$$x_{i-1} \equiv \pm 1 \pmod{n}.$$

Notice that we are **not** testing if $x_{i-1} = \pm 1$. This is because each term x_i is calculated as,

$$x_i = x_{i-1}^2 \pmod{n},$$

and by convention $0 \leq x_{i-1}^2 \pmod{n} \leq n - 1$, i.e. we give the remainder as a number in the range $[0, n - 1]$. Since,

$$n - 1 \equiv -1 \pmod{n},$$

to test if,

$$x_{i-1} \equiv \pm 1 \pmod{n},$$

we actually need to check if $x_{i-1} = 1$ or $n - 1$.

Finally, we are ready to write down an algorithm that implements the Miller-Rabin primality test and the pseudocode is given below.

```

1  /* Input: Odd n>3 being tested for primality using MR.
2   Input: k, number of tests */
3  function MillerRabinRandomizedPrimality( n, k ) {
4   /* Factor n-1 as 2^s.t, where t is odd */
5   s = 0, t = n-1;
6   while (t is even) {
7     s = s+1; t = t/2
8   } // at this stage, n-1 will be 2^s.t, where t is odd
9   /*conduct k FLT tests*/
10  loop (k times) {
11    a = choose random number in [2...n-2];
12    compute x_0      /*x_0 = a^t mod n*/
13    if (x_0 == 1) continue /*case 4: slide 34 */
14    loop (i in [1...s]) {
15      compute x_i;    /*x_i = (x_{i-1})^2 mod n */
16      if (x_i == 1) { /*x_i = a^{2^i}t mod n = 1*/
17        if (x_{i-1} != n-1) return "Composite"; /*case 2: slide 34 */
18        else {x_s = 1; break;} /*case 3: slide 34 */
19      }
20    }
21    if (x_s != 1 modulo n) return "Composite!"; /*case 1: slide 34 */
22  }
23  return "probably_prime"; // accuracy depends on k
24 }

```

Note that in the pseudocode, for any give odd n we trial k witnesses before declaring n prime. Note, $n = 3$ has to be handled separately since in Miller-Rabin's method witnesses are selected the range $[2, n - 1]$, which would be an empty range for $n = 3$. Finally, we don't actually show the computation of the terms in the sequence $\langle x_0, x_1, \dots, x_s \rangle$, but they can be computed via repeated squaring until x_s is obtained and then stored so that the appropriate primality testing may be performed.

Accuracy of Miller-Rabin

We close this section by noting that Miller-Rabin is still a probabilistic method and there are some composite numbers that will pass the sequence test, as well as our Fermat's little theorem test. These numbers are sometimes referred to as *strong pseudoprimes*. For example, 25 is a strong pseudoprime to the base 7 since the corresponding sequence is,

$$\begin{aligned}
 n - 1 &= 24, \quad s = 3, \quad t = 3 : \\
 x_0 &= 7^{2^0 \times t} \bmod 25 = 7^3 \bmod 25 = 18 \\
 x_1 &= 18^2 \bmod 25 = 24 \\
 x_2 &= 24^2 \bmod 25 = 1 \\
 x_3 &= 1^2 \bmod 25 = 1.
 \end{aligned}$$

Fortunately, there are no composite numbers which are strong pseudoprimes to all bases. In other words, there are no composite numbers that pass the Miller-Rabin test for all possible witnesses. In fact, there is no analog to the Carmichael numbers for the Miller-Rabin test, and for a given odd composite n , there are at most $\frac{n}{4}$ witnesses $0 < a < n$ for which n is a strong pseudoprime. Thus, if we choose a random witness we expect the probability of having chosen a witness for which n is a strong pseudoprime, to be at most $\frac{1}{4}$. Using this insight it can be proven that if we select k witnesses, the algorithm declares a composite number

incorrectly prime with a probability of at most $\frac{1}{4^k}$.